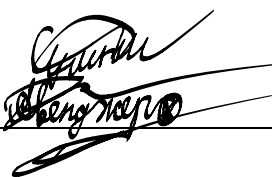


**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра САУ**

**ОТЧЕТ**  
**по лабораторной работе №8**  
**по дисциплине «Микропроцессорная техника в мехатронике и**  
**робототехнике»**  
**ТЕМА: протокол передачи данных UART**

Студент гр. 2491

Преподаватель



---

---

Сулинский Е.В.  
Недовизий А.И.

Копычев М.М.

Санкт-Петербург

2025

### **Цель работы:**

Познакомиться с протоколом передачи UART и научиться передавать данные по данному протоколу.

### **Задание на работу:**

Необходимо написать программу, которая позволяет осуществлять двухсторонний обмен данными между МК и компьютером. Согласно схеме организовать возможность управления яркостью светодиода с терминала компьютера, а также организовать «эхо» посылаемых с терминала на МК данных, представляющих собой задание коэффициента заполнения ШИМ сигнала яркости светодиода.

### **Краткие теоретические сведения:**

Помимо уже рассмотренных нами ранее протоколов обмена SPI (работа 7) и TWI (работа 6), почти каждый МК имеет встроенный модуль универсального последовательного интерфейса – UART (или более обще – USART, что расшифровывается как Universal Synchronous/Asynchronous Receiver-Transmitter или Универсальный Синхронно-Асинхронный ПриёмоПередачик УСАПП). AVR тут не исключение и поддерживает этот протокол в полном объёме полностью аппаратно (на уровне регистров). По структуре протокол UART (для простоты и краткости будем называть его именно так) – это асинхронный последовательный протокол, то есть передающая сторона (master или slave) по очереди выдаёт в линию логический 0 и 1, а принимающая (slave или master, соответственно) отслеживает их и «запоминает». Синхронизация идет по времени – приемник и передатчик заранее договариваются о том, на какой частоте будет идти обмен. Это очень важный момент – если скорость передатчика и приемника не будут совпадать, то передачи может не быть вообще, либо будут считаны не те данные.

Временная диаграмма протокола передачи UART представлена на рисунке 1:

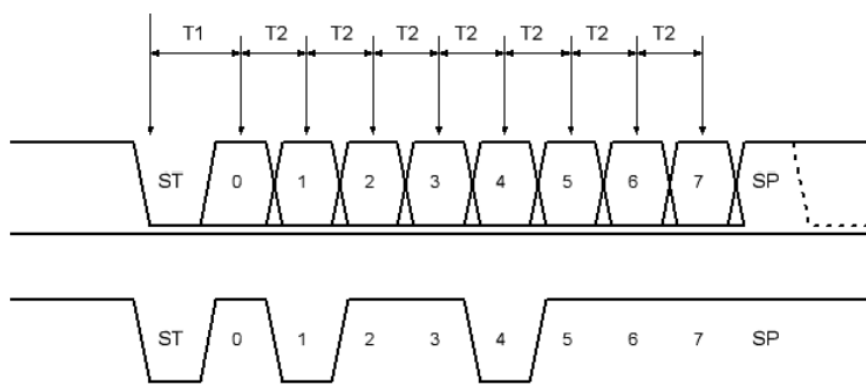


Рисунок 1 – временная диаграмма протокола передачи UART

Для аппаратной организации обмена данными по протоколу UART следует подключить выводы Rx и Tx МК к выводам Tx и Rx модуля UARTUSB (именно так – перекрёстно – Rx одного к Tx другого и наоборот, как показано на рисунке 2).

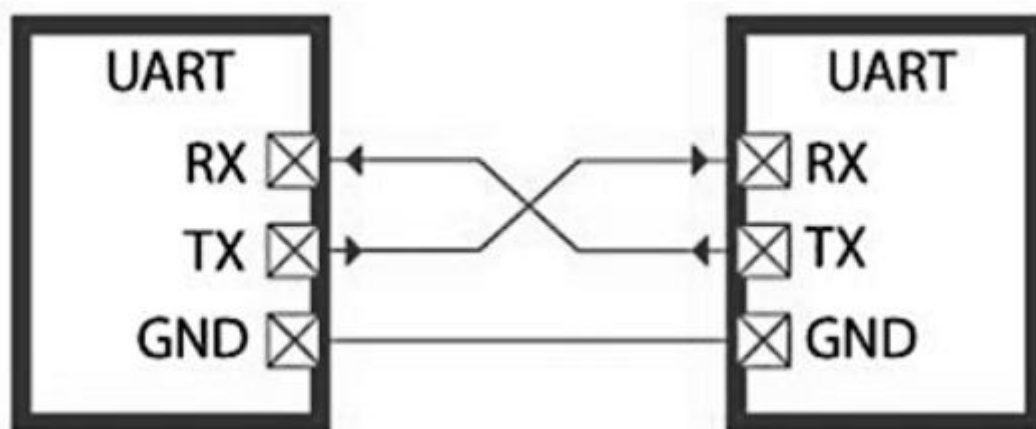


Рисунок 2 – перекрёстное подключение интерфейсных проводов UART

## Ход работы

Напишем код программы для реализации задания. Составим блок-схемы для наглядного представления алгоритма работы.

```
#include <avr/io.h>
```

```
#define F_CPU 11059200UL
```

```
#include <util/delay.h>
```

```
#include <avr/interrupt.h>
```

```
const uint8_t segments[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
```

```
uint16_t pulseWidth = 0; // Объявление переменной pulseWidth
```

```
int8_t Digit(int32_t d, uint8_t m)
```

```
{
```

```
    int8_t i = 5, a;
```

```
    while (i)
```

```
    {        // цикл по разрядам числа
```

```
        a = d % 10; // выделяем очередной разряд
```

```
        if (i-- == m)
```

```
            break; // выделен заданный разряд - уходим
```

```
        d /= 10; // уменьшаем число в 10 раз
```

```
    }
```

```
    return (a);
```

```
}
```

```
void timer_init()
```

```
{
```

```
    DDRE |= (1 << 3) | (1 << 4) | (1 << 5);
```

```
    TCCR3A = (1 << COM3A1) | (1 << COM3B1) | (1 << COM3C1) | (1 << WGM30) | (1 << WGM31);
```

```
    TCCR3B = (1 << WGM32) | (1 << CS31);
```

```
}
```

```
void LED(uint16_t value)
```

```
{
```

```
    OCR3A = 255 - value * 2.55 ;
```

```
    OCR3B = 255;
```

```
    OCR3C = 255 - value * 2.55;
```

```
}
```

```
void init_segments()
```

```
{
```

```
    DDRA = 0b00111110;
```

```
    DDRC = 0xFF;
```

```
}
```

```
void generate_impulse()
```

```
{
```

```
    PORTE |= (1 << 6);
```

```
    _delay_us(15);
```

```
    PORTE &= ~(1 << 6);
```

```
    _delay_ms(100);
```

```
}
```

```
void display(uint16_t dist)
```

```
{
```

```
    uint8_t i = 0;
```

```
    while (Digit(dist, i) == 0) // с помощью этого цикла мы убираем нули
```

```
    {
```

```
        PORTA |= (1 << i);
```

```
        PORTC = 0x0;
```

```

    _delay_us(20); // пауза
    PORTA &= ~(1 << i);
    i++; // переход в следующий разряд
}

while (i <= 5) // запоминает i при котором у нас первая не нулевая цифра и выводит
последовательно оставшиеся цифры в разряды
{
    PORTC = segments[Digit(dist, i)];
    PORTA |= (1 << i);
    // подача в регистр нужной последовательности, чтобы выбрать нужный разряд
    _delay_us(20); // подача в регистр нужной последовательности, чтобы выбрать
нужный разряд
    PORTA &= ~(1 << i); // ставим регистр в изначальное значение
    i++; // переход в следующий разряд
}
}

uint16_t dist;

int main(void)
{
    timer_init();
    init_segments();

    TCCR1A = 0;
    TCCR1B = (1 << CS11);

    DDRE |= (1 << 6);
    EIMSK = (1 << INT7);
    EICRB = (1 << ISC70);
    sei();

```

```

while (1)
{
    generate_impulse();
    LED(dist);
    display(dist);
}
}

ISR(INT7_vect)
{
    if ((PINE & (1 << 7)) != 0)
    {
        // Запись начала в старшую часть
        TCNT1H = 0;
        // Запись в младшую часть
        TCNT1L = 0;
    }
    else
    {
        // Чтение начала младшей части
        pulseWidth = TCNT1L;
        // Чтение старшей части
        pulseWidth |= (TCNT1H << 8);
    }

    dist = (float)pulseWidth/52; // примерно в см
    if (dist < 10)
        dist = 10;
    if (dist > 100)
        dist = 100;
}-

```

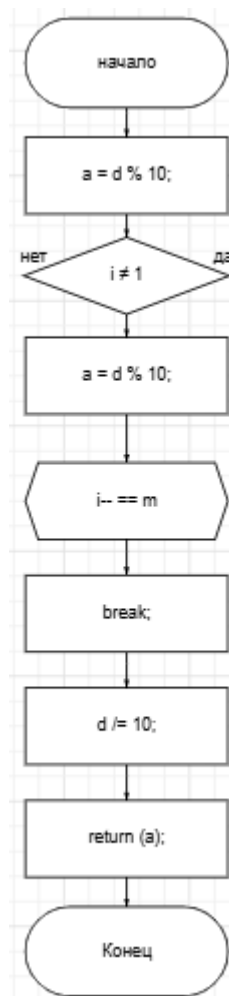


Рисунок 3 – “Блок-схема функции Digit(int32\_t d, uint8\_t m)”

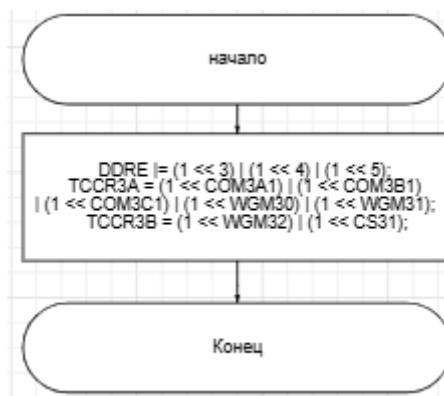


Рисунок 4 – “Блок-схема функции timer\_init()”



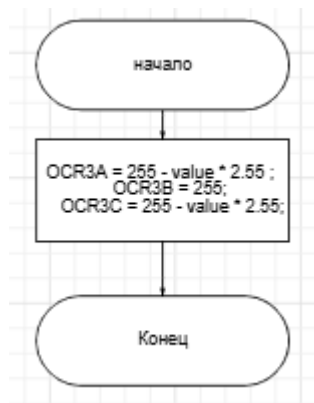


Рисунок 5 – Блок-схема функции LED(uint16\_t value)

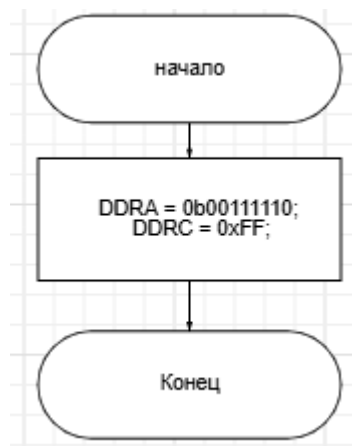


Рисунок 6 – Блок-схема функции init\_segments()

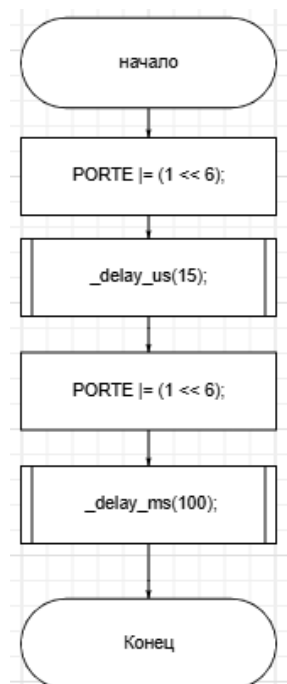


Рисунок 7 – Блок-схема функции generate\_impulse()

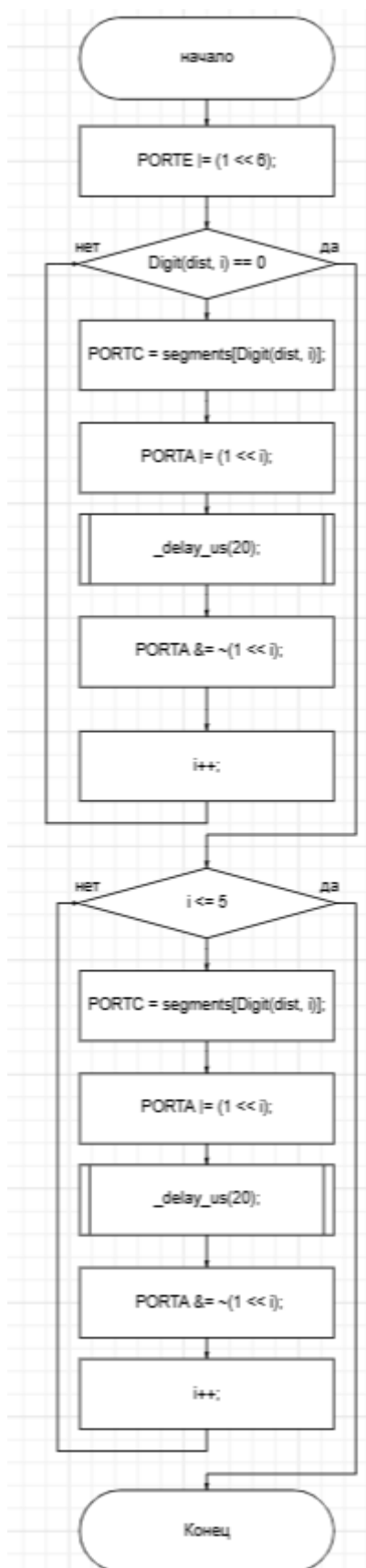


Рисунок 8 – Блок-схема функции display(uint16\_t dist)

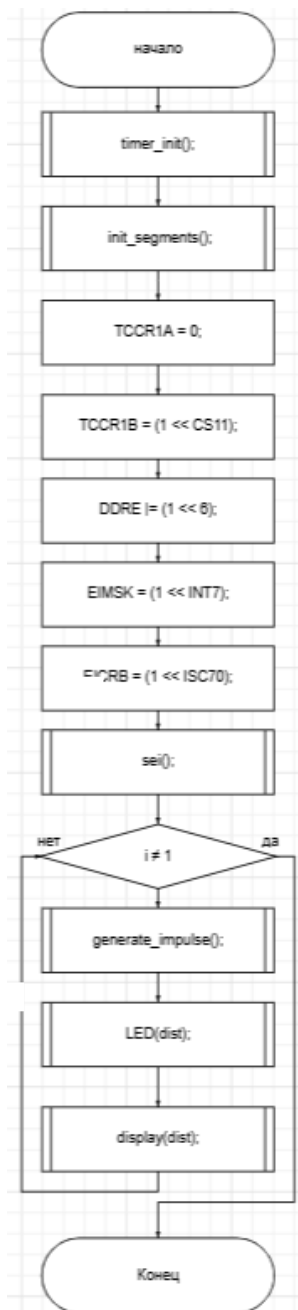


Рисунок 9 – “Блок-схема функции `int main(void)`”

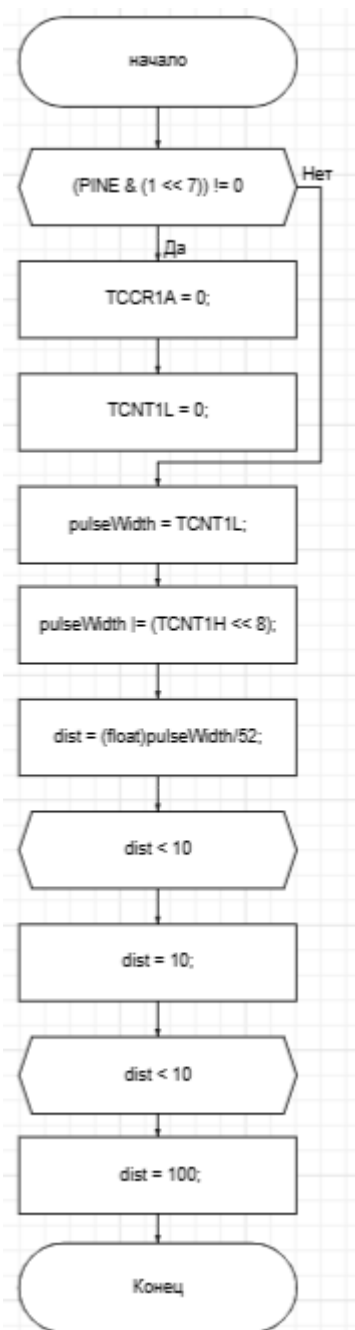


Рисунок 10 – “Блок-схема функции ISR(INT7\_vect)”

### Выводы:

В ходе выполнения данной лабораторной работы мы познакомились с протоколом передачи данных UART и научились передавать данные по данному протоколу. Также была написана программа, которая позволяет осуществлять двухсторонний обмен данными между микроконтроллером и компьютером.